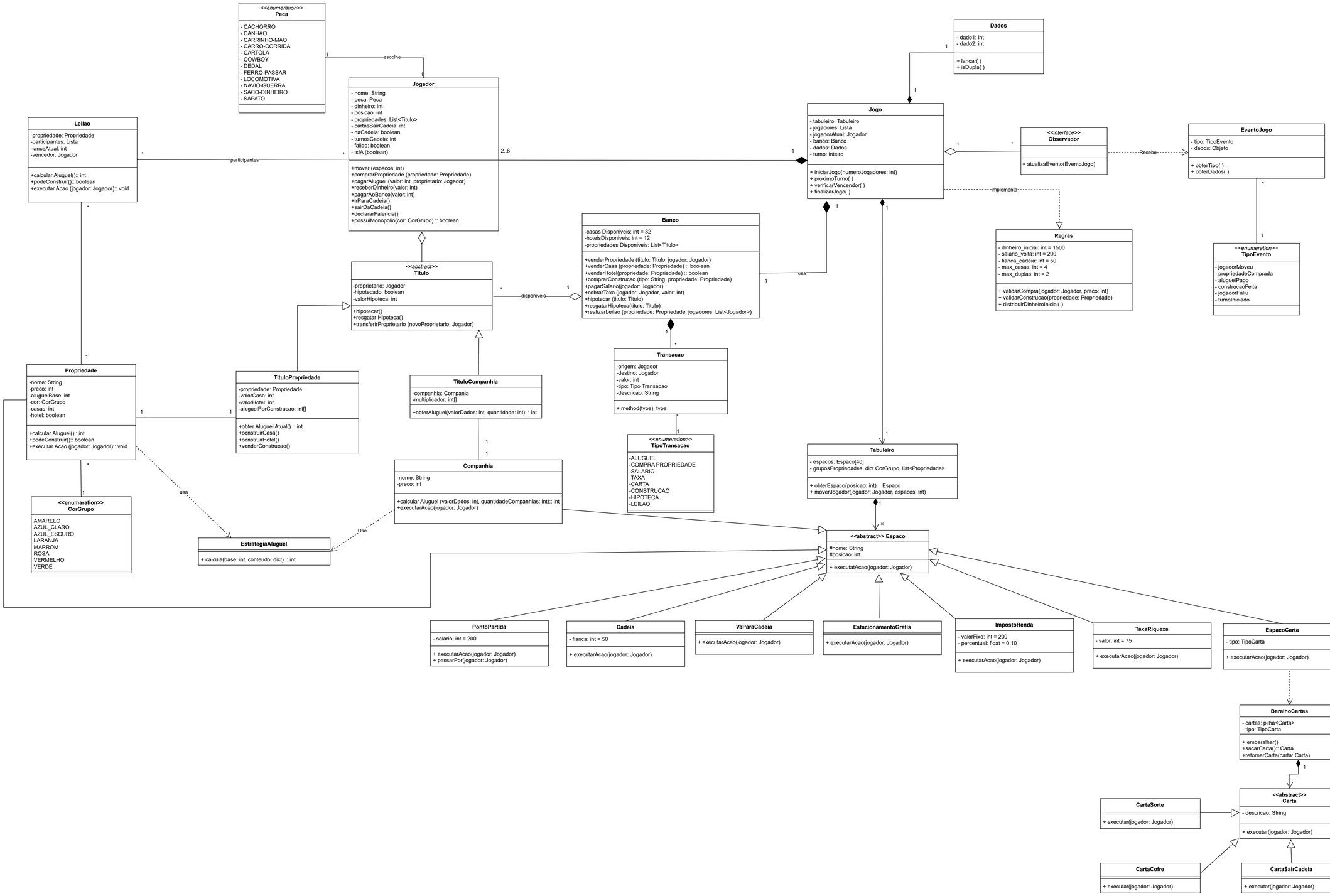

<<enumeration>>
Peca
- CACHORRO
- CANHAO
- CARRINHO-MAO
- CARRO-CORRIDA
- CARTOLA
- COWBOY
- DEDAL
- FERRO-PASSAR
- LOCOMOTIVA
- NAVIO-GUERRA
- SACO-DINHEIRO
- SAPATO
escolhe
Jogador
- nome: String
- peca: Peca
- dinheiro: int
- posicao: int
- propriedades: List<Titulo>
- cartasSairCadeia: int
- naCadeia: boolean
- turnosCadeia: int
- falido: boolean
- isIA (boolean)
+mover (espacos: int)
+comprarPropriedade (propriedade: Propriedade)
+pagarAluguel (valor: int, proprietario: Jogador)
+receberDinheiro(valor: int)
+pagarAoBanco(valor: int)
+irParaCadeia()
+sairDaCadeia()
+declararFalencia()
+possuiMonopolio(cor: CorGrupo) :: boolean
Leilao
-propriedade: Propriedade
-participantes: Lista
-lanceAtual: int
-vencedor: Jogador
+calcular Aluguel():: int
+podeConstruir():: boolean
+executar Acao (jogador: Jogador):: void
participantes
2..6
Dados
- dado1: int
- dado2: int
+ lancar()
+ isDupla()
Jogo
- tabuleiro: Tabuleiro
- jogadores: Lista
- jogadorAtual: Jogador
- banco: Banco
- dados: Dados
- turno: inteiro
+ iniciarJogo(numeroJogadores: int)
+ proximoTurno()
+ verificarVencendor()
+ finalizarJogo()
<<interface>>
Observador
+ atualizaEvento(EventoJogo)
Recebe
EventoJogo
- tipo: TipoEvento
- dados: Objeto
+ obterTipo()
+ obterDados()
implementa
Regras
- dinheiro_inicial: int = 1500
- salario_volta: int = 200
- fianca_cadeia: int = 50
- max_casas: int = 4
- max_duplas: int = 2
+ validarCompra(jogador: Jogador, preco: int)
+ validarConstrucao(propriedade: Propriedade)
+ distribuirDinheiroInicial()
<<enumeration>>
TipoEvento
- jogadorMoveu
- propriedadeComprada
- aluguelPago
- construcaoFeita
- jogadorFaliu
- turnoIniciado
Banco
-casas Disponiveis: int = 32
-hoteisDisponiveis: int = 12
-propriedades Disponiveis: List<Titulo>
+venderPropriedade (titulo: Titulo, jogador: Jogador)
+venderCasa (propriedade: Propriedade) :: boolean
+venderHotel(propriedade: Propriedade) :: boolean
+comprarConstrucao (tipo: String, propriedade: Propriedade)
+pagarSalario(jogador: Jogador)
+cobrarTaxa (jogador: Jogador, valor: int)
+hipotecar (titulo: Titulo)
+resgatarHipoteca(titulo: Titulo)
+realizarLeilao (propriedade: Propriedade, jogadores: List<Jogador>)
usa
<<abstract>>
Titulo
-proprietario: Jogador
-hipotecado: boolean
-valorHipoteca: int
+hipotecar()
+resgatar Hipoteca()
+transferirProprietario (novoProprietario: Jogador)
disponiveis
Transacao
-origem: Jogador
-destino: Jogador
-valor: int
-tipo: Tipo Transacao
-descricao: String
+ method(type): type
<<enumeration>>
TipoTransacao
-ALUGUEL
-COMPRA PROPRIEDADE
-SALARIO
-TAXA
-CARTA
-CONSTRUCAO
-HIPOTECA
-LEILAO
Propriedade
-nome: String
-preco: int
-aluguelBase: int
-cor: CorGrupo
-casas: int
-hotel: boolean
+calcular Aluguel():: int
+podeConstruir():: boolean
+executar Acao (jogador: Jogador):: void
TituloPropriedade
-propriedade: Propriedade
-valorCasa: int
-valorHotel: int
-aluguelPorConstrucao: int[]
+obter Aluguel Atual() :: int
+construirCasa()
+construirHotel()
+venderConstrucao()
TituloCompanhia
-companhia: Compania
-multiplicador: int[]
+obterAluguel(valorDados: int, quantidade: int): : int
Companhia
-nome: String
-preco: int
+calcular Aluguel (valorDados: int, quantidadeCompanhias: int):: int
+executarAcao(jogador: Jogador)
<<enumaration>>
CorGrupo
AMARELO
AZUL_CLARO
AZUL_ESCURO
LARANJA
MARROM
ROSA
VERMELHO
VERDE
usa
EstrategiaAluguel
+ calcula(base: int, conteudo: dict) :: int
Use
Tabuleiro
- espacos: Espaco[40]
- gruposPropriedades: dict CorGrupo, list<Propriedade>
+ obterEspaco(posicao: int): : Espaco
+ moverJogador(jogador: Jogador, espacos: int)
40
<<abstract>> Espaco
#nome: String
#posicao: int
+ executatAcao(jogador: Jogador)
PontoPartida
- salario: int = 200
+ executarAcao(jogador: Jogador)
+ passarPor(jogador: Jogador)
Cadeia
- fianca: int = 50
+ executarAcao(jogador: Jogador)
VaParaCadeia
+ executarAcao(jogador: Jogador)
EstacionamentoGratis
+ executarAcao(jogador: Jogador)
ImpostoRenda
- valorFixo: int = 200
- percentual: float = 0.10
+ executarAcao(jogador: Jogador)
TaxaRiqueza
- valor: int = 75
+ executarAcao(jogador: Jogador)
EspacoCarta
- tipo: TipoCarta
+ executarAcao(jogador: Jogador)
BaralhoCartas
- cartas: pilha<Carta>
- tipo: TipoCarta
+ embaralhar()
+sacarCarta():: Carta
+retornarCarta(carta: Carta)
<<abstract>>
Carta
- descricao: String
+ executar(jogador: Jogador)
<<enumaration>>
TipoCarta
SORTE
COFRE
CartaSorte
+ executar(jogador: Jogador)
CartaCofre
+ executar(jogador: Jogador)
CartaSairCadeia
+ executar(jogador: Jogador)